



Politechnika
Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

Badanie efektywności algorytmów grafowych w zależności od rozmiaru instancji oraz sposobu reprezentacji grafu w pamięci komputera

Imię i nazwisko: Wiktor Zięba

Numer albumu: 281056

Wrocław, czerwiec 2025

Spis treści

1	Wstęp	2
1.1	Algorytm Prima	2
1.2	Algorytm Kruskala	3
1.3	Algorytm Dijkstry	4
1.4	Algorytm Bellmana-Forda	5
2	Plan eksperymentu	6
3	Wyznaczanie najkrótszych ścieżek	7
3.1	Tabele z wynikami pomiarów	7
3.2	Wykresy - typ1	9
3.3	Wykresy - typ2	12
3.4	Wnioski	17
4	Wyznaczanie minimalnego drzewa spinającego	17
4.1	Tabele z wynikami pomiarów	17
4.2	Wykresy - typ1	19
4.3	Wykresy - typ2	22
4.4	Wnioski	27
5	Literatura	27

1 Wstęp

Zadanie projektowe polegało na zaimplementowaniu struktur grafów w postaci macierzy incydencji oraz listy sąsiedztwa, a następnie dokonaniu pomiarów i porównaniu między sobą czasów wykonania poszczególnych algorytmów realizujących dwa podstawowe zagadnienia teorii grafów

- **Wyznaczania minimalnego drzewa spinającego:**

- algorytm Prima,
- algorytm Kruskala

- **Wyznaczania najkrótszych ścieżek w grafie:**

- algorytm Dijkstry,
- algorytm Bellmana-Forda.

1.1 Algorytm Prima

Algorytm prima to algorytm zachłanny wyznaczający minimalne drzewo rozpinające w grafie nieskierowanym, spójnym, posiadającym krawędzie z wagami. Algorytm startuje z dowolnie wybranego wierzchołka, a następnie w każdej iteracji wybiera krawędź o najniższej wadze, która łączy nowy wierzchołek (nie należący do MST) ze zbudowanym drzewem. Proces ten jest powtarzany do momentu aż wszystkie wierzchołki zostaną dołączone do drzewa rozpinającego.

Złożoność obliczeniowa algorytmu Prima zależy od reprezentacji grafu oraz zastosowanej struktury danych.

Table 1: Złożoność czasowa algorytmu Prima

Struktura danych	Złożoność czasowa (łącznie)
Macierz sąsiedztwa, liniowe przeszukiwanie	$\mathcal{O}(V ^2)$
Lista sąsiedztwa z kopcem binarnym	$\mathcal{O}((V + E) \log V) = \mathcal{O}(E \log V)$
Lista sąsiedztwa z kopcem Fibonacciego	$\mathcal{O}(E + V \log V)$

Table 2: Złożoność pamięciowa algorytmu Prima

Reprezentacja grafu	Złożoność pamięciowa
Macierz incydencji	$\mathcal{O}(V * E)$
Lista sąsiedztwa	$\mathcal{O}(V + E)$

W projekcie zaimplementowano wersję algorytmu Prima z kopcem binarnym na reprezentacjach macierzy incydencji oraz listy sąsiedztwa.

```

1 Prim(graf)
2   klucz[0..V-1] := INF
3   wMST[0..V-1] := false
4
5   klucz[0] := 0
6   kopiec := MinHeap()
7   kopiec.push(0,0)
8
9   suma := 0
10  while kopiec niepusty:
11      (koszt, u) := usun minimum z kopca
12      if inMST[u] == true:
13          pomin
14      wMST[u] := true
15      suma += koszt
16  for (u, v, w) w graf:
17      if !inMST[v] and w < key[v]:
18          key[v] := w
19          kopiec.push(w, v)
20
21  return suma

```

Tekst 1: Algorytm Prim pseudokod

1.2 Algorytm Kruskala

Algorytm Kruskala to algorytm zachłanny służący do wyznaczania minimalnego drzewa rozpinającego w grafie nieskierowanym, spójnym. Algorytm początkowo traktuje każdy wierzchołek jako osobny zbiór i sortuje wszystkie krawędzie rosnąco według wag. Następnie w kolejnych iteracjach dodaje do drzewa krawędzie, które w wyniku dodania do drzewa łączą dwa zbiory - czyli włączają do drzewa wierzchołek nie tworzący cyklu, aż do momentu, gdy MST zawiera dokładnie $V - 1$ krawędzi.

Złożoność czasowa algorytmu Kruskala wynosi $\mathcal{O}(E \log E)$. Wynika to z konieczności posortowania krawędzi. Całkowita złożoność pamięciowa algorytmu to $\mathcal{O}(V + E)$.

Table 3: Złożoność czasowa algorytmu Kruskala

Reprezentacja grafu	Złożoność czasowa
Macierz incydencji	$\mathcal{O}(V^2 + E \log E)$
Lista sąsiedztwa	$\mathcal{O}(E \log E)$

Table 4: Złożoność pamięciowa algorytmu Kruskala

Reprezentacja grafu	Złożoność pamięciowa
Macierz incydencji	$\mathcal{O}(V * E)$
Lista sąsiedztwa	$\mathcal{O}(V + E)$

```

1 Kruskal(graf):
2
3     krawedzie := graf.getEdges()
4     posortuj(krawedzie)
5
6     zainicjalizuj zbiory rozlaczne dla kazdego wierzchołka
7     suma := 0
8
9     for (u, v, w) in krawedzie:
10         jesli find(u) != find(v):
11             union(u, v)
12             suma += w
13
14     return suma

```

Tekst 2: Algorytm Kruskala pseudokod

1.3 Algorytm Dijkstry

Algorytm Dijkstry to zachłanny algorytm znajdujący najkrótsze ścieżki od podanego wierzchołka startowego do wszystkich pozostałych wierzchołków. Warunkiem działania algorytmu Dijkstry jest założenie, że wszystkie krawędzie w grafie posiadają wagi nieujemne. Algorytm w każdej iteracji wybiera wierzchołek dla którego obecny dystans od wierzchołka startowego jest najmniejszy. Następnie aktualizowane są odległości do jego sąsiadów. Podstawową operacją w algorytmie Dijkstry jest relaksacja - jest to proces aktualizacji szacowanej odległości do danego wierzchołka w grafie. Relaksacja jest wykonywana dla każdej krawędzi wychodzącej z aktualnie przetwarzanego wierzchołka. Równanie relaksacji zapisuje się w postaci:

$$\text{jeśli } d[v] > d[u] + w(u, v), \quad \text{to } d[v] \leftarrow d[u] + w(u, v), \quad \pi[v] \leftarrow u$$

Złożoność obliczeniowa Dijkstry zależy od użytej struktury oraz od reprezentacji grafu. Dla najbardziej optymalnej implementacji algorytmu stosując kopiec binarny oraz reprezentację listową złożoność czasowa algorytmu wynosi $\mathcal{O}((V + E) \log V)$, w przypadku reprezentacji macierzy incydencji, każda iteracja wymaga sprawdzenia wszystkich potencjalnych krawędzi wychodzących co pogarsza ogólną złożoność do $\mathcal{O}(V^2)$.

Table 5: Złożoność czasowa algorytmu Dijkstry

Reprezentacja grafu	Złożoność czasowa
Lista sąsiedztwa i kopiec binarny	$\mathcal{O}((V + E) \log V)$
Macierz incydencji	$\mathcal{O}(V^2)$

Table 6: Złożoność pamięciowa algorytmu Dijkstry

Reprezentacja grafu	Złożoność pamięciowa
Lista sąsiedztwa	$\mathcal{O}(V + E)$
Macierz incydencji	$\mathcal{O}(V \cdot E)$

```

1 Dijkstra(graf, start):
2
3     dist[0..V-1] := INF
4     odwiedzone[0..V-1] := false
5     dist[start] := 0
6
7     kopiec := MinHeap()
8     kopiec.push(0, start)
9
10    while kopiec niepusty:
11        (koszt, u) := kopiec.popMin()
12        if u juz odwiedzony:
13            pomin
14        odwiedzone[u] := true
15
16        for (v, w) in sasiedzi(u):
17            if not odwiedzone[v] and dist[u] + w < dist[v]:
18                dist[v] := dist[u] + w
19                kopiec.push(dist[v], v)
20
21    return dist

```

Tekst 3: Algorytm Dijkstry pseudokod

1.4 Algorytm Bellmana-Forda

Algorytm Bellmana-Forda to algorytm wyszukiwania najkrótszych ścieżek od podanego wierzchołka startowego do pozostałych wierzchołków w grafie. W odróżnieniu od algorytmu Dijkstry, algorytm jest w stanie wyznaczyć najkrótsze ścieżki w grafie, w którym znajdują się krawędzie o ujemnych wagach, lub wskazać istnienie cyklu ujemnego w grafie. Czyni to algorytm Bellmana-Forda wolniejszym od Dijkstry z uwagi na konieczność wykonania do $V - 1$ iteracji relaksujących krawędzie. Po wykonaniu wszystkich iteracji wykonywana jest jeszcze dodatkowa pętla sprawdzająca, czy nadal zmieniają się dystanse, jeżeli tak zwracana jest informacja o istnieniu w grafie cyklu ujemnego zamiast wyniku.

Złożoność czasowa algorytmu Bellmana-Forda to $\mathcal{O}(V \cdot E)$, wynika to z faktu wykonywania $V - 1$ iteracji, w każdej wykonując relaksację. Dla gęstych grafów złożoność może spaść nawet do rzędu $\mathcal{O}(V^3)$.

Table 7: Złożoność czasowa algorytmu Bellmana-Forda

Reprezentacja grafu	Złożoność czasowa
Lista sąsiedztwa	$\mathcal{O}(V \cdot E)$
Macierz incydencji	$\mathcal{O}(V \cdot E)$

Table 8: Złożoność pamięciowa algorytmu Bellmana-Forda

Reprezentacja grafu	Złożoność pamięciowa
Lista sąsiedztwa	$\mathcal{O}(V + E)$
Macierz incydencji	$\mathcal{O}(V \cdot E)$

```

1 BellmanFord(graf, start):
2
3     dist[0..V-1] := INF
4     dist[start] := 0
5
6     for i in 0..V-1:
7         for (u, v, w) in krawedzie:
8             if dist[u] + w < dist[v]:
9                 dist[v] := dist[u] + w
10                pi[v] := u
11
12     for (u, v, w) in krawedzie:
13         if dist[u] + w < dist[v]:
14             return "ujemny cykl"
15
16     return dist

```

Tekst 4: Algorytm Bellmana-Forda pseudokod

2 Plan eksperymentu

Celem eksperymentu było zaimplementowanie dwóch struktur reprezentacji grafu: macierzy incydencji oraz listy sąsiedztwa. Oraz algorytmów realizujących dwa podstawowe problemy grafów: wyznaczania minimalnego drzewa spinającego (Prim i Kruskal) oraz wyznaczania najkrótszych ścieżek w grafie (Dijkstra i Bellman-Ford) Należało wykonać pomiary czasów wykonywania się poszczególnych algorytmów w zależności od:

- liczby wierzchołków w grafie
- gęstości grafu
- sposobu reprezentacji grafu

Pomiary wykonano dla grafów o liczbie wierzchołków kolejno: 400, 500, 600, 700, 800, 900, 1000 oraz gęstości: 0.25, 0.5, 0.99. Wagi generowanych krawędzi mieściły się w zakresie od 1 do 100. Wykonano 50 powtórzeń wszystkich pomiarów, generując za każdym razem nowe grafy. Wyniki z wszystkich powtórzeń zostały uśrednione. Dla problemu MST generowano grafy nieskierowane, spójne a dla problemu najkrótszych ścieżek grafy skierowane. Dla poszczególnych problemów zachowano wspólny graf w celu zapewnienia porównywalnych warunków testowych. Zawartość wygenerowanego grafu w formie macierzowej w każdej iteracji była kopiowana do grafu listy sąsiedztwa zamiast generować nowy graf. Dla problemu wyznaczania najkrótszych ścieżek w grafie początkowy wierzchołek jest losowany w każdym powtórzeniu.

Wszelkie pomiary czasów wykonania poszczególnych algorytmów zostały zrealizowane przy użyciu `std::chrono::high_resolution_clock`. Pomiary obejmowały jedynie czasy wykonania algorytmów z pominięciem generowania danych. Jednostką w której podane zostały wszystkie wyniki jest milisekunda. Wszystkie testy przeprowadzone zostały na tym samym komputerze w celu zapewnienia spójnych warunków pomiarowych. Aby zapewnić spójność generowanego grafu najpierw budowano drzewo spinające, a dopiero potem losowano pozostałe krawędzi aż do uzyskania docelowej gęstości.

3 Wyznaczanie najkrótszych ścieżek

3.1 Tabele z wynikami pomiarów

Table 9: Czasy działania algorytmu Bellman–Ford – lista sąsiedztwa

V	Gęstość	Czas [ms]
400	0.25	51.6849
400	0.50	103.2540
400	0.99	205.6190
500	0.25	101.1710
500	0.50	202.1270
500	0.99	395.9650
600	0.25	169.2090
600	0.50	340.4100
600	0.99	686.0140
700	0.25	274.3330
700	0.50	548.3250
700	0.99	1099.4200
800	0.25	399.5750
800	0.50	808.2550
800	0.99	1615.4200
900	0.25	577.5240
900	0.50	1160.0700
900	0.99	2287.6700
1000	0.25	771.8130
1000	0.50	1565.4700
1000	0.99	3170.5700

Table 10: Czasy działania algorytmu Bellman–Ford – macierz incydencji

V	Gęstość	Czas [ms]
400	0.25	50.4770
400	0.50	100.0880
400	0.99	198.7330
500	0.25	98.0182
500	0.50	196.0410
500	0.99	386.3840
600	0.25	167.3700
600	0.50	334.7520
600	0.99	669.9710
700	0.25	269.5990
700	0.50	537.4440
700	0.99	1075.2200
800	0.25	393.8490
800	0.50	796.5880
800	0.99	1592.8600
900	0.25	567.5880
900	0.50	1143.8100
900	0.99	2249.7800
1000	0.25	762.4860
1000	0.50	1544.0600
1000	0.99	3130.1200

Table 11: Czasy działania algorytmu Dijkstra – reprezentacja: lista sąsiedztwa

V	Gęstość	Czas [ms]
400	0.25	2.01292
400	0.50	4.81484
400	0.99	9.33266
500	0.25	3.14478
500	0.50	7.25322
500	0.99	17.02050
600	0.25	5.57664
600	0.50	13.10340
600	0.99	23.07470
700	0.25	5.81796
700	0.50	13.60610
700	0.99	27.24820
800	0.25	8.55098
800	0.50	18.40510
800	0.99	38.91230
900	0.25	11.07910
900	0.50	23.36940
900	0.99	49.31680
1000	0.25	15.15280
1000	0.50	29.37420
1000	0.99	66.84890

Table 12: Czasy działania algorytmu Dijkstra – reprezentacja: macierz incydencji

V	Gęstość	Czas [ms]
400	0.25	32.48910
400	0.50	64.94910
400	0.99	132.22700
500	0.25	65.05160
500	0.50	128.30800
500	0.99	251.30100
600	0.25	110.73100
600	0.50	221.90000
600	0.99	426.81200
700	0.25	170.68700
700	0.50	335.36600
700	0.99	661.73400
800	0.25	253.27500
800	0.50	497.53600
800	0.99	980.83800
900	0.25	355.35200
900	0.50	700.36700
900	0.99	1388.65000
1000	0.25	485.07200
1000	0.50	958.27500
1000	0.99	1906.51000

3.2 Wykresy - typ1

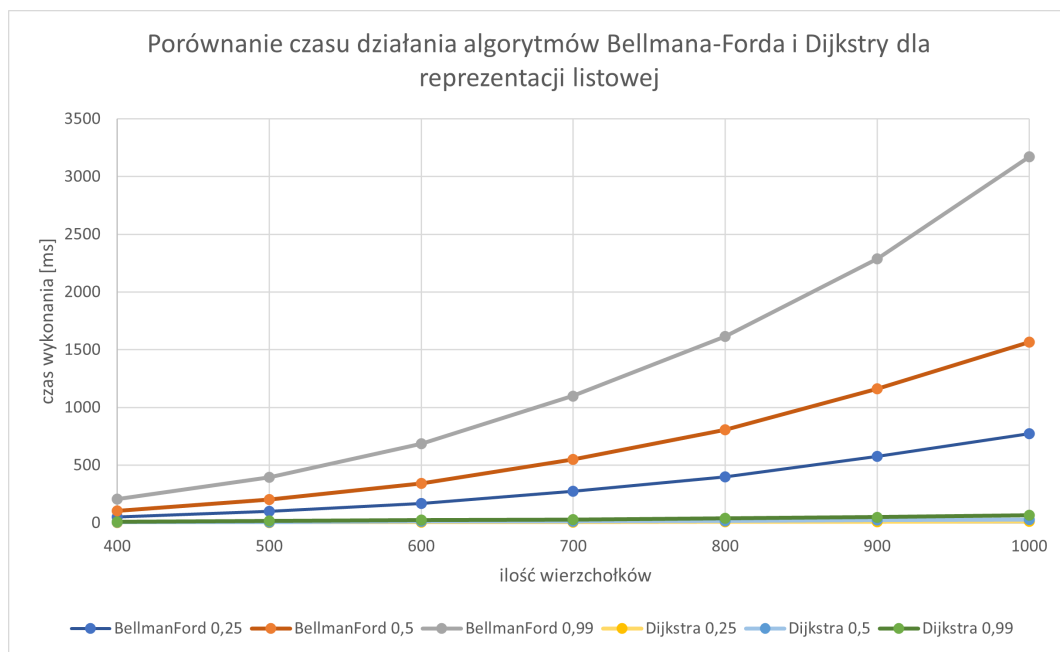


Figure 1: Czasy Dijkstry i Bellmana-Forda – lista sąsiedztwa (typ 1).

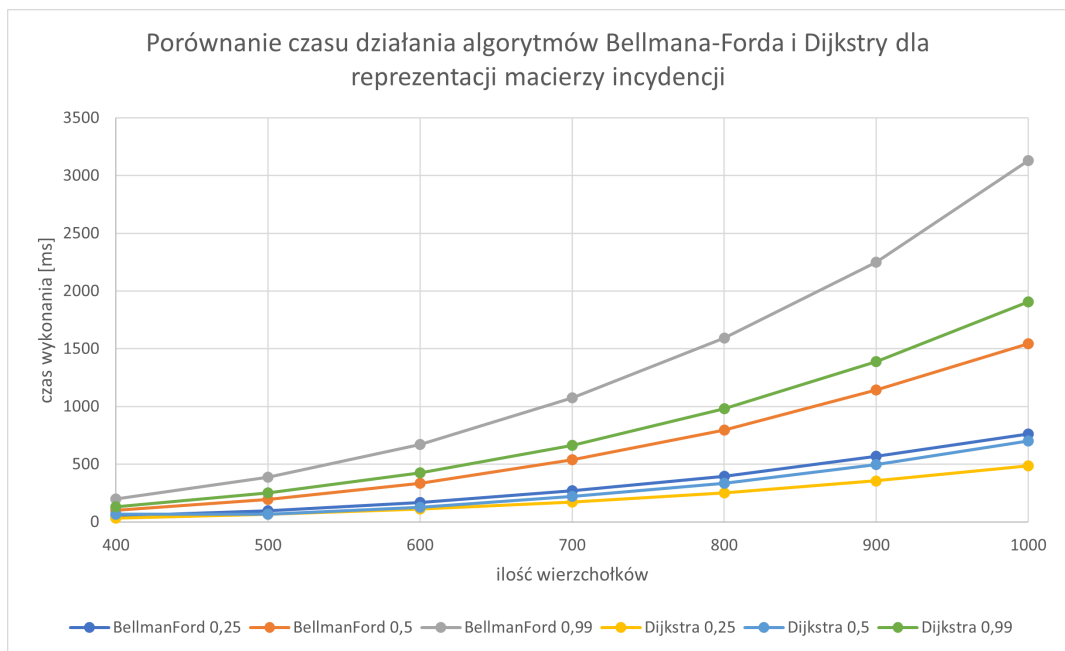


Figure 2: Czasy Dijkstry i Bellmana-Forda – macierz incydencji (typ 1).

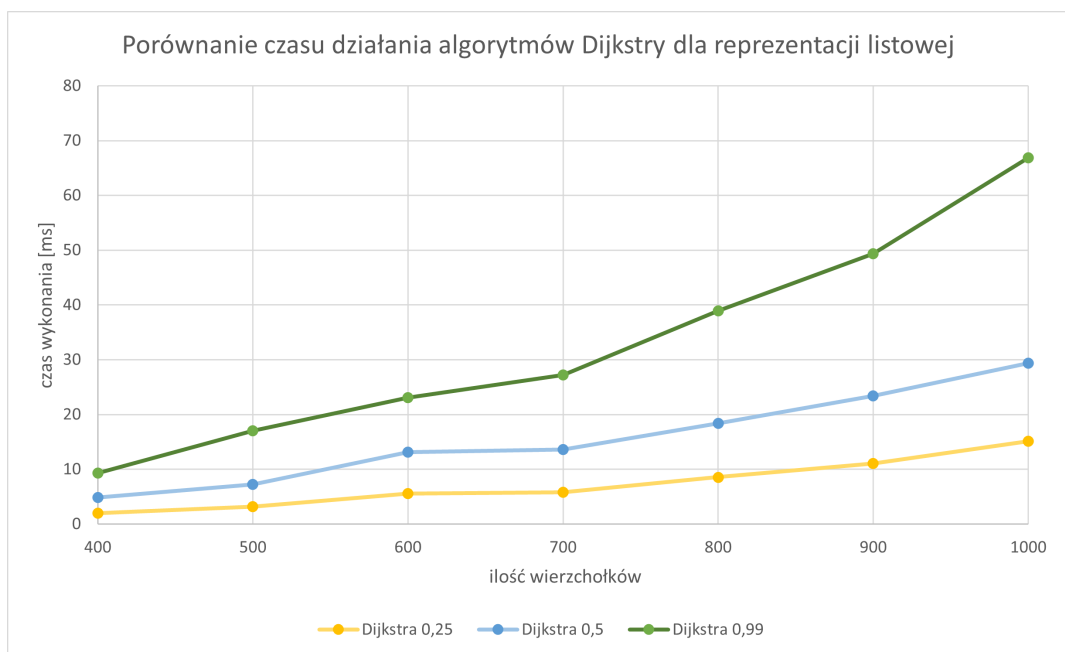


Figure 3: Algorytm Dijkstra – lista sąsiedztwa (typ 1).

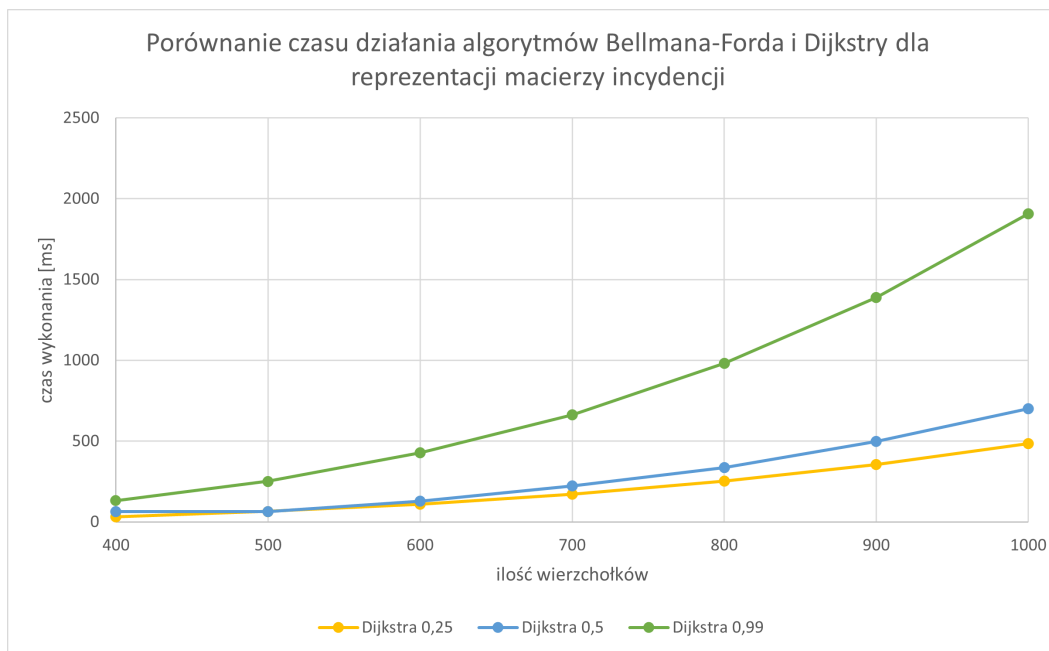


Figure 4: Algorytm Dijkstra – macierz incydencji (typ 1).

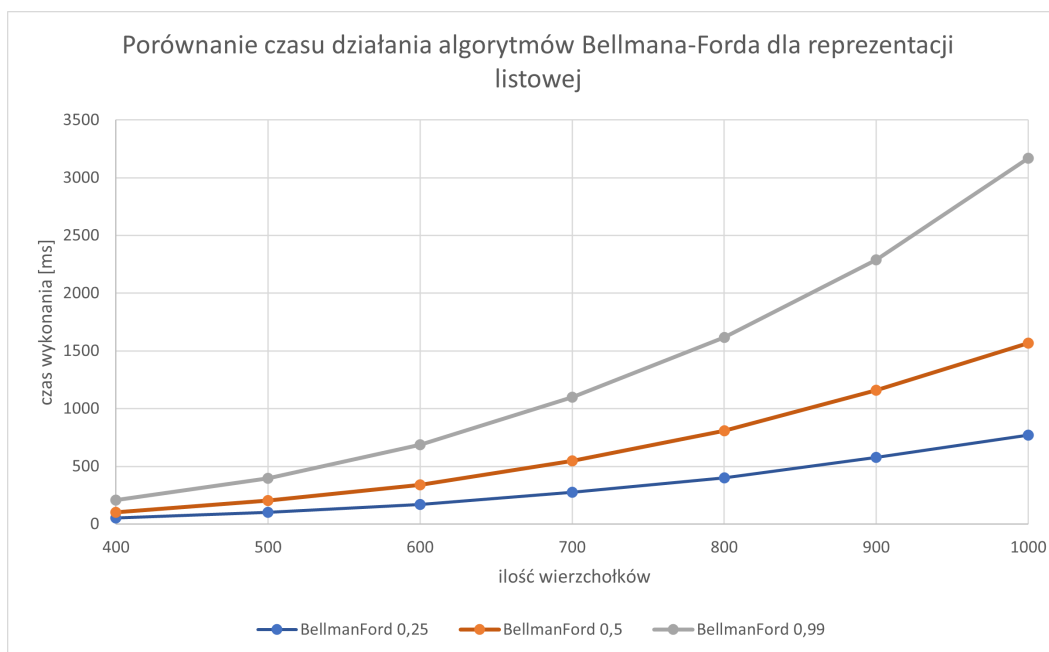


Figure 5: Algorytm Bellman-Ford – lista sąsiedztwa (typ 1).

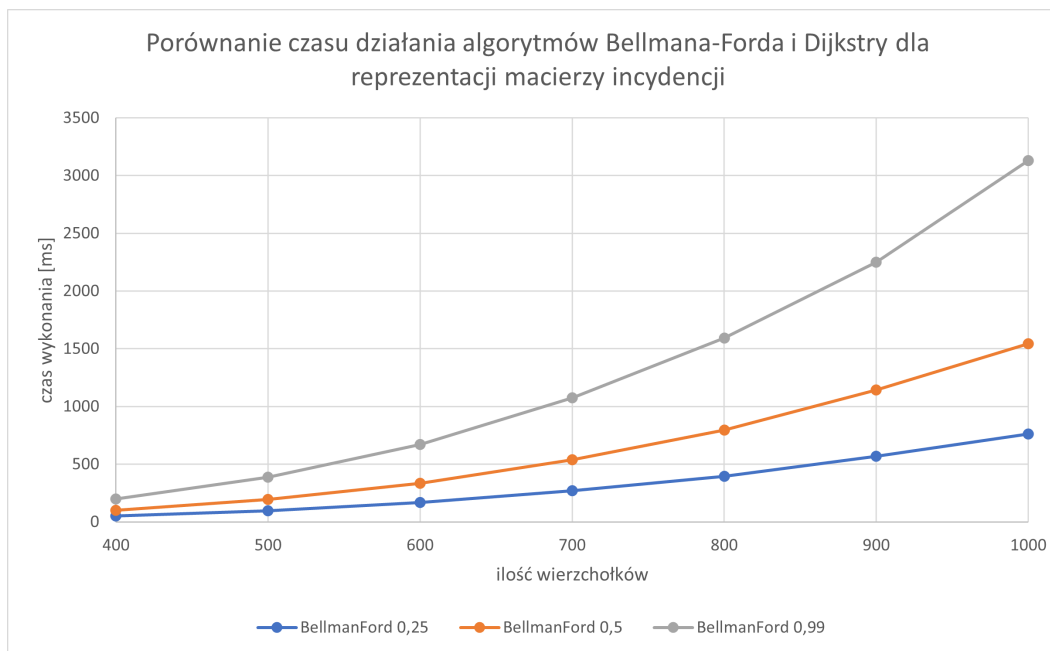


Figure 6: Algorytm Bellman-Ford – macierz incydencji (typ 1).

3.3 Wykresy - typ2

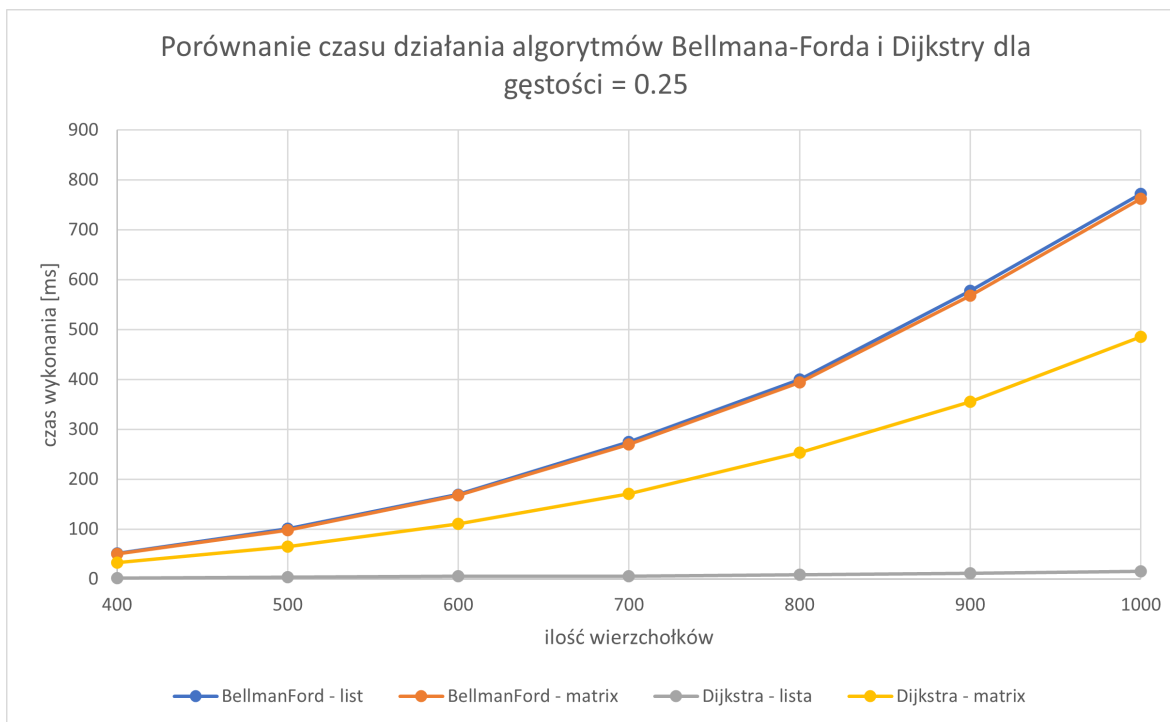


Figure 7: Czasy Dijkstra i Bellman-Ford – gęstość 25 %, lista vs macierz.

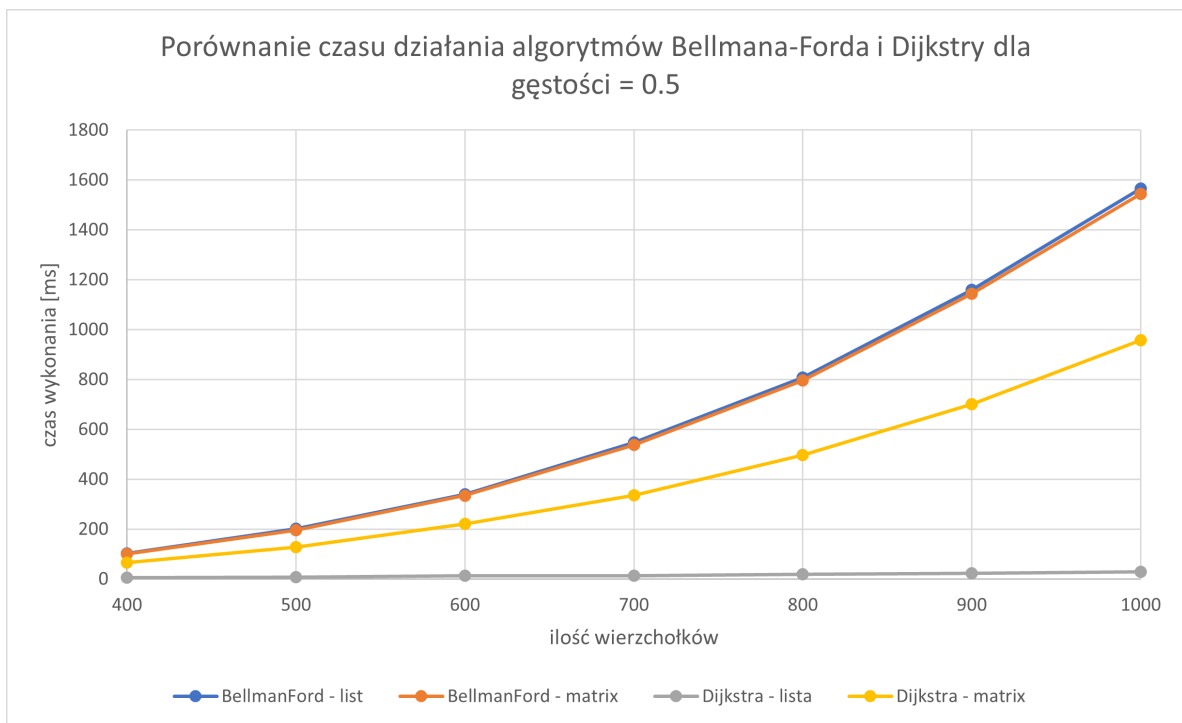


Figure 8: Czasy Dijkstra i Bellman-Ford – gęstość 50 %, lista vs macierz.

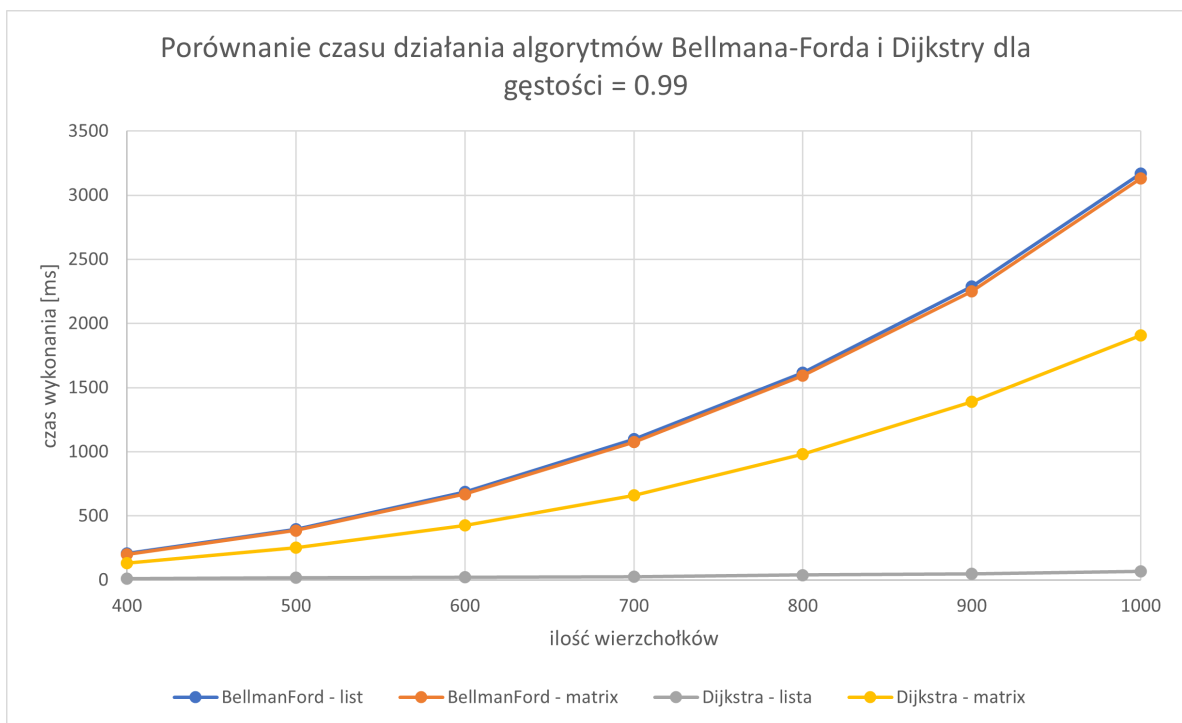


Figure 9: Czasy Dijkstra i Bellman-Ford – gęstość 99 %, lista vs macierz.

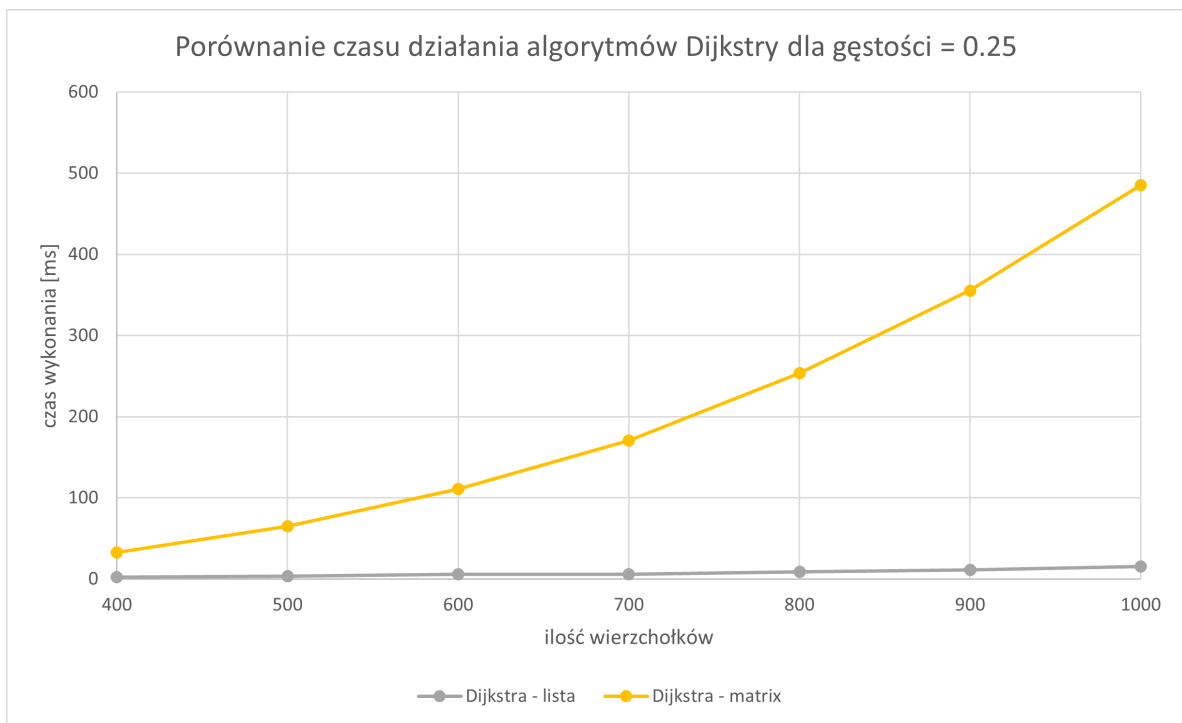


Figure 10: Dijkstra – gęstość 25 %, lista vs macierz.

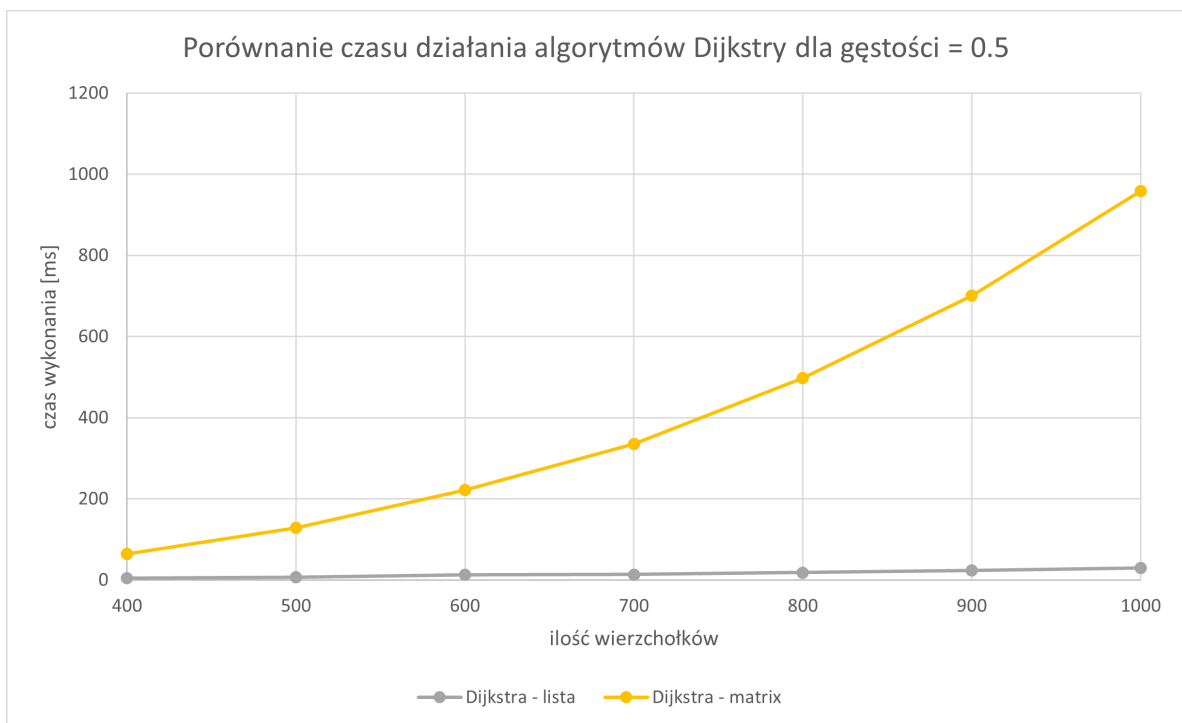


Figure 11: Dijkstra – gęstość 50 %, lista vs macierz.

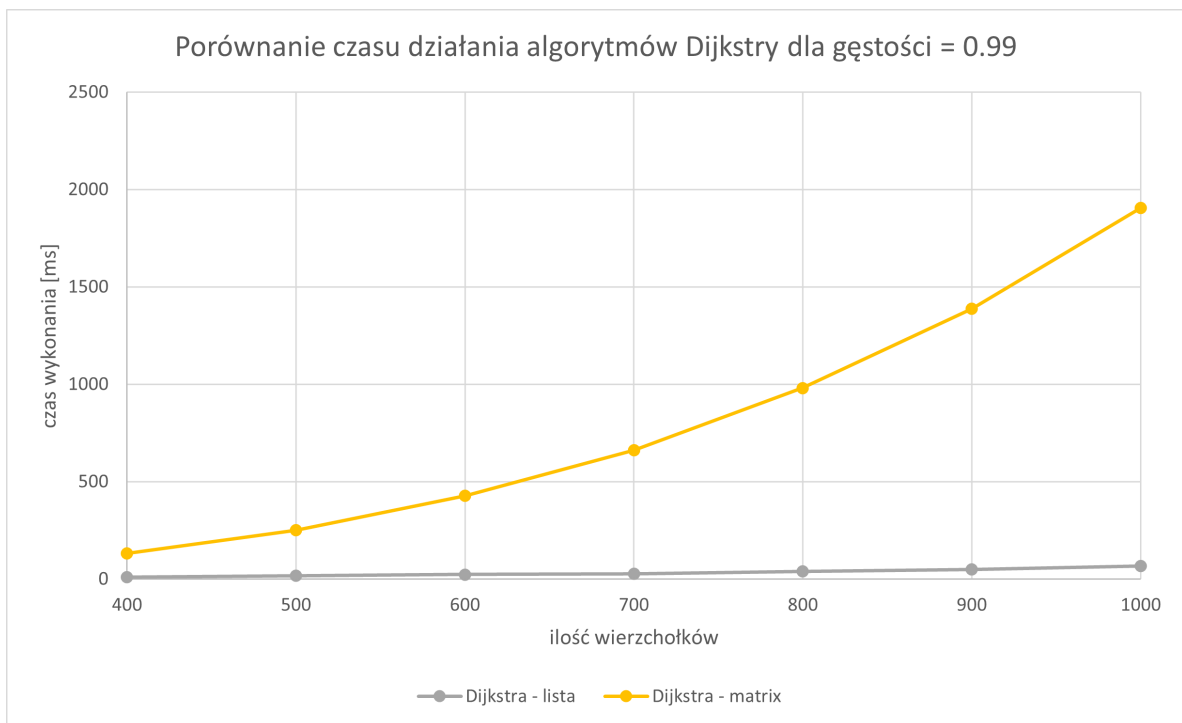


Figure 12: Dijkstra – gęstość 99 %, lista vs macierz.

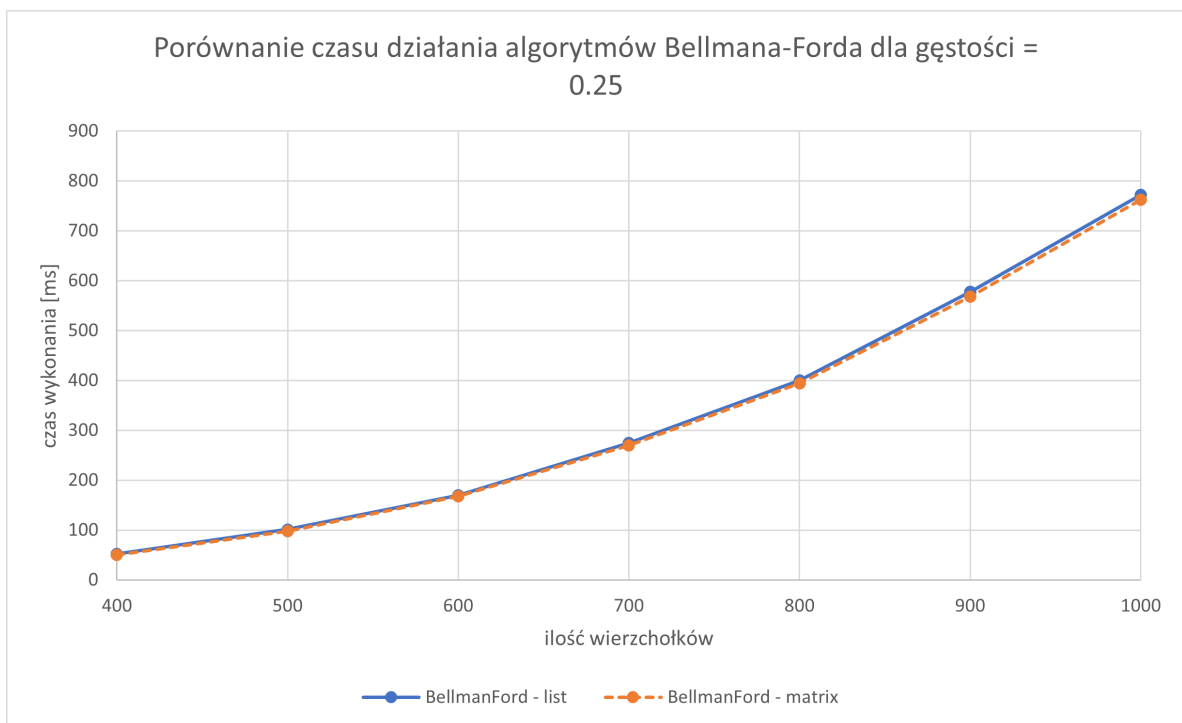


Figure 13: Bellman–Ford – gęstość 25 %, lista vs macierz.

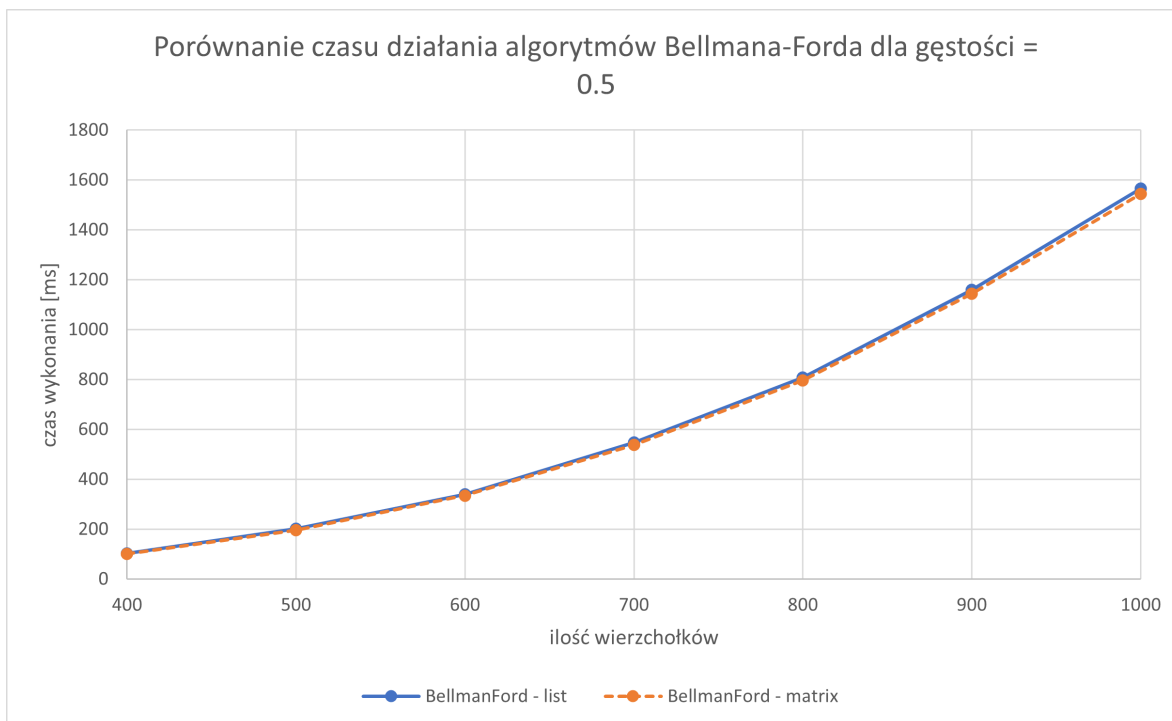


Figure 14: Bellman-Ford – gęstość 50 %, lista vs macierz.

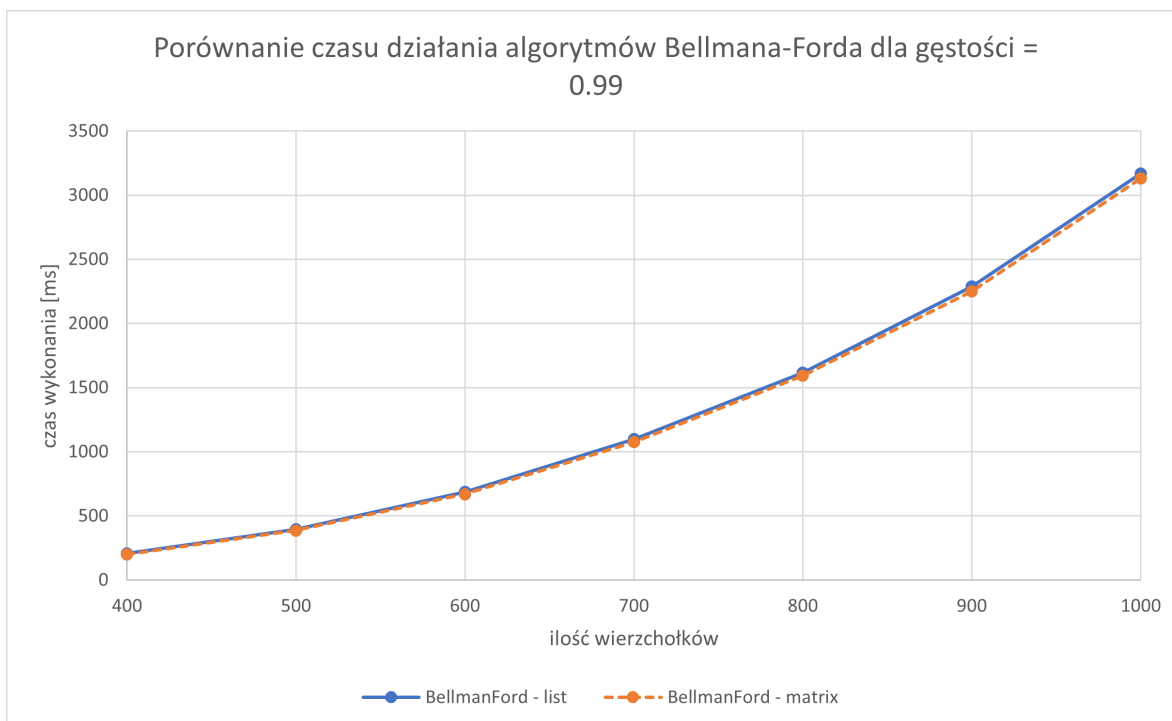


Figure 15: Bellman-Ford – gęstość 99 %, lista vs macierz.

3.4 Wnioski

Algorytm Dijkstra na grafie w reprezentacji listy sąsiedztwa okazał się - zgodnie z przewidywaniami najszybszym algorytmem znajdowania najkrótszej ścieżki spośród rozważanych. Reprezentacja macierzy incydencji znacząco pogorszyła jego wydajność - czas wykonania wzrósł nawet kilkukrotnie w porównaniu do wersji na liście sąsiedztwa. Wynika to z faktu, że każdorazowo trzeba przechodzić cały wiersz macierzy co pogarsza złożoność do $\mathcal{O}(V^2)$.

Algorytm Bellmana-Forda niezależnie od reprezentacji przegrywał z algorytmem Dijkstry. Warto uwzględnić fakt, że algorytm ten można znacząco przyspieszyć dodając warunek kończący pętlę w momencie, gdy kolejna relaksacja nic nie zmienia w odległościach. Dla założeń projektu, gdzie nie występują ujemne wagi (wagi 1-100) znacząco skróciło by to czas wykonania, jednak testy przeprowadzono bez tej optymalizacji. Bellman-Ford dla obu reprezentacji dawał porównywalne wyniki, minimalne różnice prowadzą się do implementacji struktur danych.

Podsumowując, algorytm Dijkstry na reprezentacji listowej wypada najlepiej pod względem czasu wykonania, jednak algorytm ten nie obsługuje możliwości pojawienia się ujemnych krawędzi, co czyni algorytm Bellmana-Forda użytecznym w sytuacjach gdzie takie wagi występują.

4 Wyznaczanie minimalnego drzewa spinającego

4.1 Tabele z wynikami pomiarów

Table 13: Czasy działania algorytmu Kruskal – reprezentacja: lista sąsiedztwa

V	Gęstość	Czas [ms]
400	0.25	2.45800
400	0.50	5.50292
400	0.99	11.53070
500	0.25	3.87868
500	0.50	8.93728
500	0.99	20.68210
600	0.25	5.95858
600	0.50	15.72660
600	0.99	29.37200
700	0.25	7.87510
700	0.50	16.61110
700	0.99	37.31010
800	0.25	10.53440
800	0.50	22.53520
800	0.99	55.65880
900	0.25	13.64160
900	0.50	29.38180
900	0.99	71.25970
1000	0.25	16.45390
1000	0.50	37.34880
1000	0.99	95.55790

Table 14: Czasy działania algorytmu Kruskal – reprezentacja: macierz incydencji

V	Gęstość	Czas [ms]
400	0.25	1.93132
400	0.50	3.83154
400	0.99	7.89912
500	0.25	2.95430
500	0.50	6.09282
500	0.99	12.73600
600	0.25	4.34800
600	0.50	9.00268
600	0.99	18.61130
700	0.25	5.91944
700	0.50	12.42120
700	0.99	25.51800
800	0.25	7.84200
800	0.50	16.38140
800	0.99	33.37210
900	0.25	10.16700
900	0.50	20.96450
900	0.99	42.67040
1000	0.25	12.69020
1000	0.50	25.85470
1000	0.99	53.18030

Table 15: Czasy działania algorytmu Prim – reprezentacja: lista sąsiedztwa

V	Gęstość	Czas [ms]
400	0.25	1.92624
400	0.50	4.64188
400	0.99	9.92316
500	0.25	3.31986
500	0.50	8.03548
500	0.99	17.99910
600	0.25	5.05534
600	0.50	13.24420
600	0.99	24.54610
700	0.25	6.29430
700	0.50	14.88600
700	0.99	29.29210
800	0.25	8.94802
800	0.50	18.05420
800	0.99	41.79630
900	0.25	12.90100
900	0.50	25.05360
900	0.99	50.51830
1000	0.25	15.43210
1000	0.50	31.20700
1000	0.99	68.21170

Table 16: Czasy działania algorytmu Prim – reprezentacja: macierz incydencji

V	Gęstość	Czas [ms]
400	0.25	18.14720
400	0.50	36.43910
400	0.99	72.99070
500	0.25	34.83670
500	0.50	70.14860
500	0.99	142.18400
600	0.25	61.26380
600	0.50	124.58800
600	0.99	241.47200
700	0.25	95.25580
700	0.50	186.88000
700	0.99	373.83200
800	0.25	147.24000
800	0.50	279.50900
800	0.99	547.22200
900	0.25	197.05000
900	0.50	393.03700
900	0.99	770.12700
1000	0.25	267.87800
1000	0.50	530.46200
1000	0.99	1057.71000

4.2 Wykresy - typ1

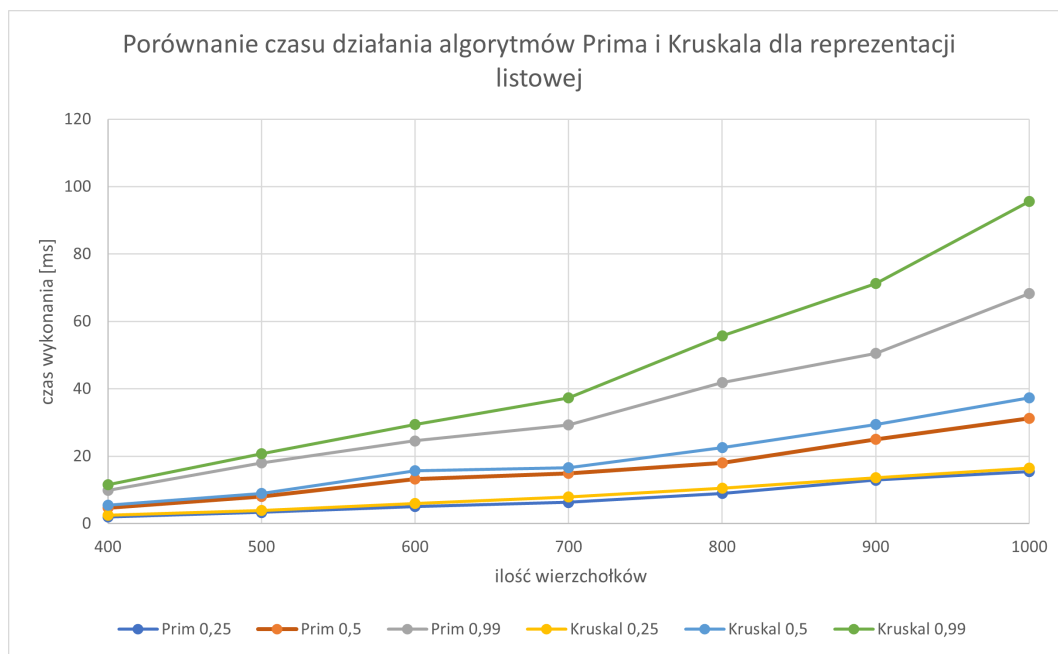


Figure 16: Czasy Prima i Kruskala – lista sąsiedztwa (typ 1).

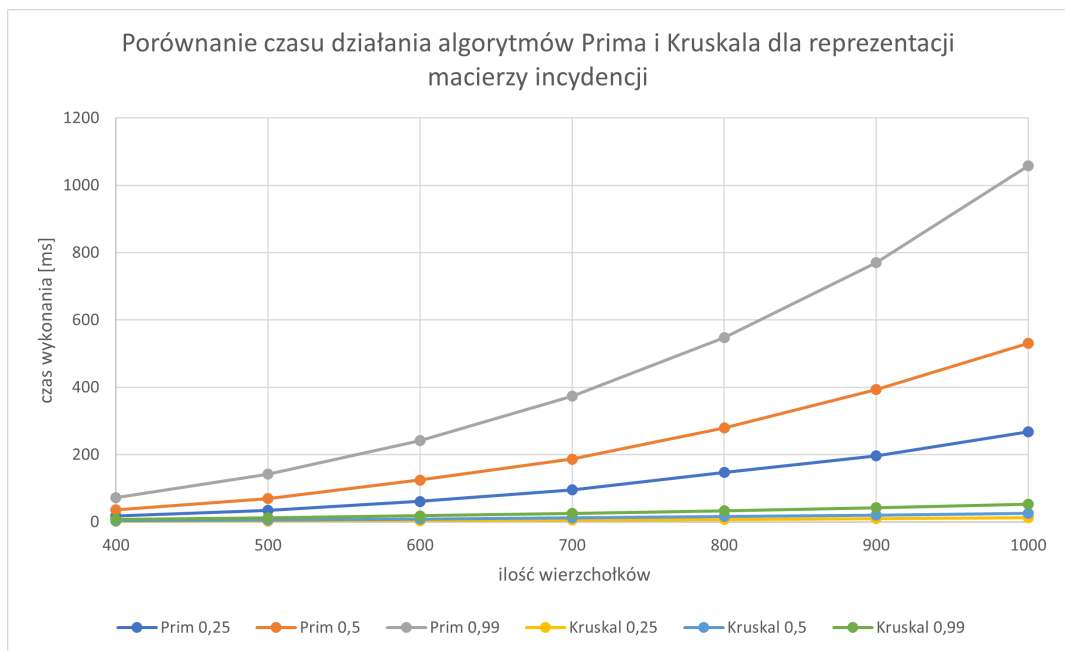


Figure 17: Czasy Prima i Kruskala – macierz incydencji (typ 1).

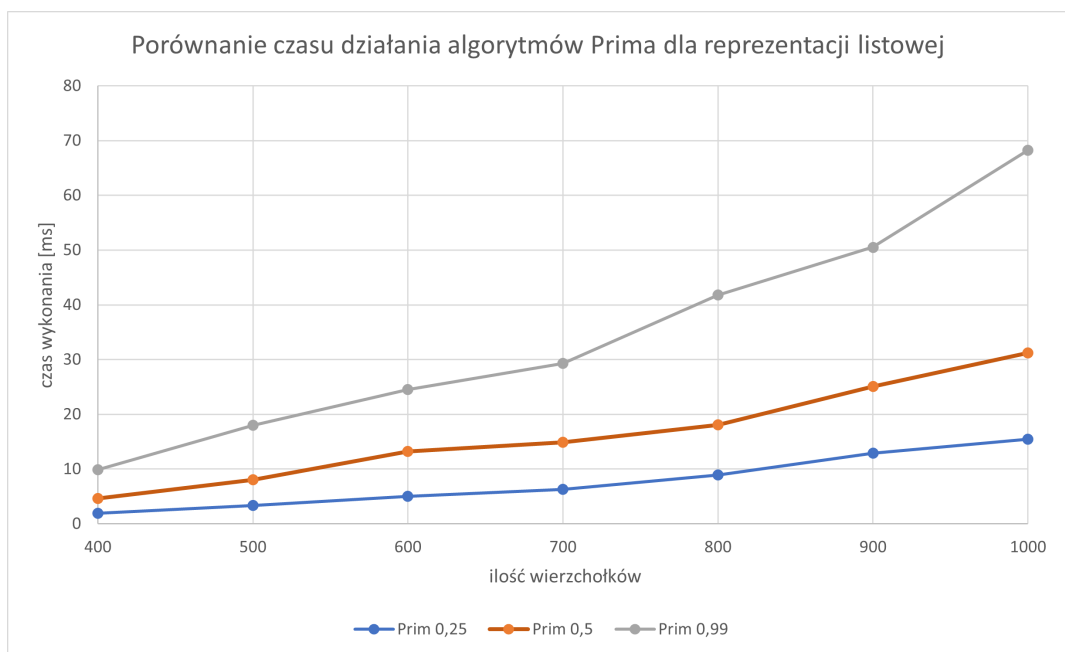


Figure 18: Algorytm Prima – lista sąsiedztwa (typ 1).

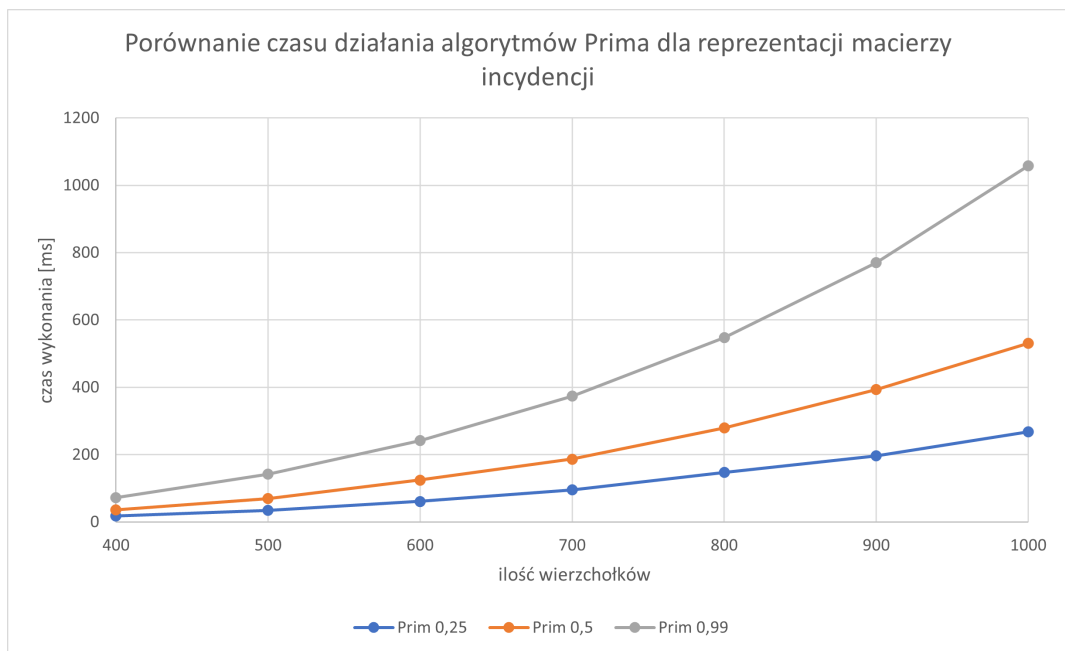


Figure 19: Algorytm Prima – macierz incydencji (typ 1).

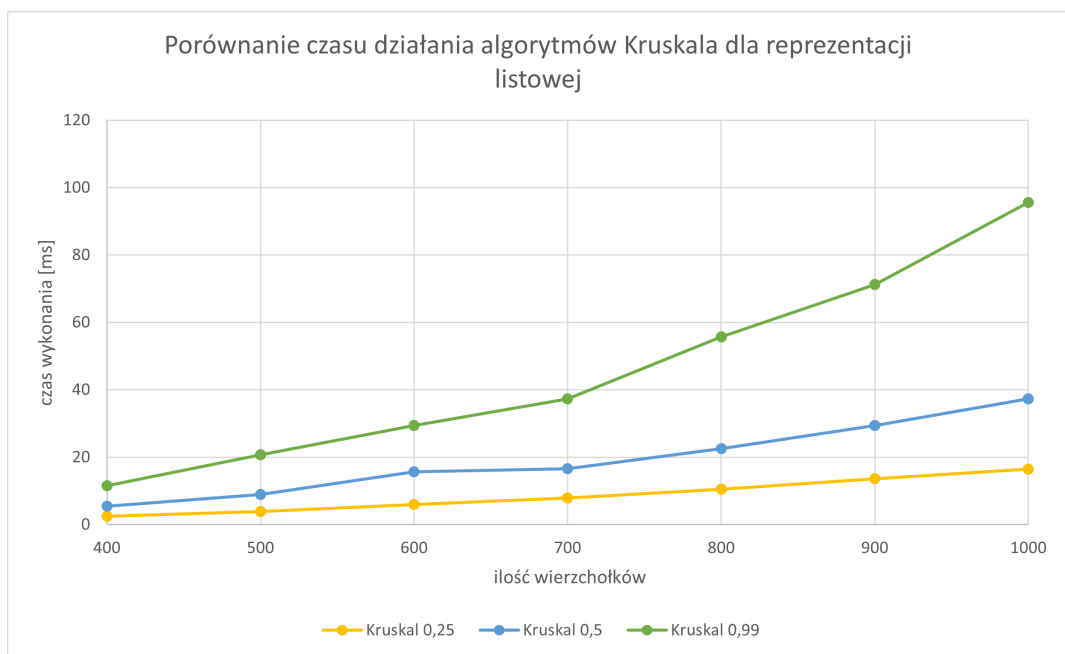


Figure 20: Algorytm Kruskal – lista sąsiedztwa (typ 1).

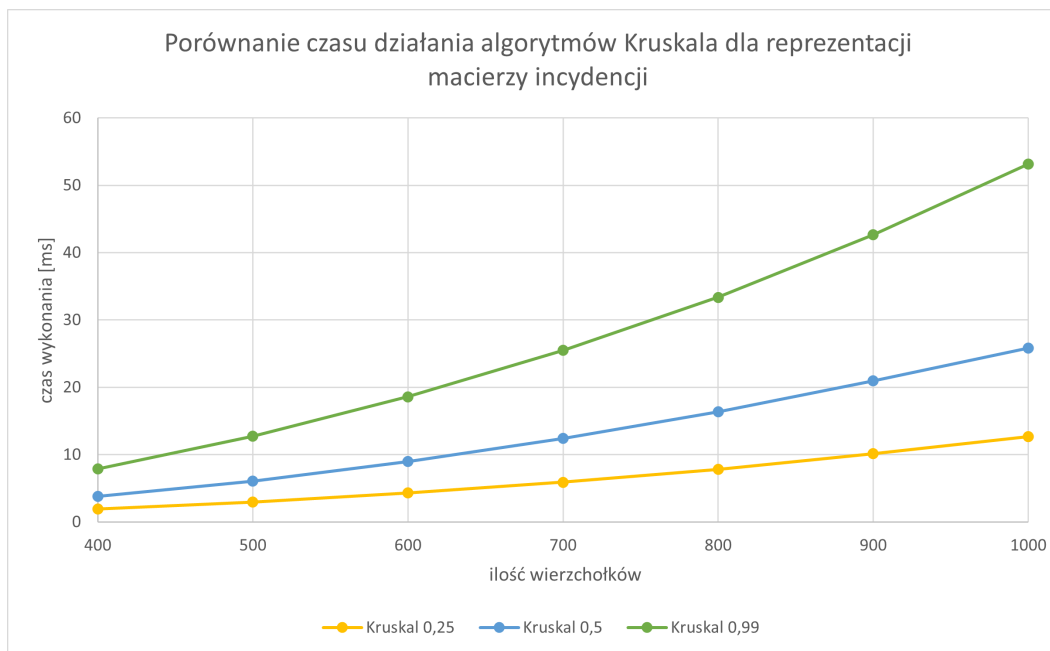


Figure 21: Algorytm Kruskal – macierz incydencji (typ 1).

4.3 Wykresy - typ2

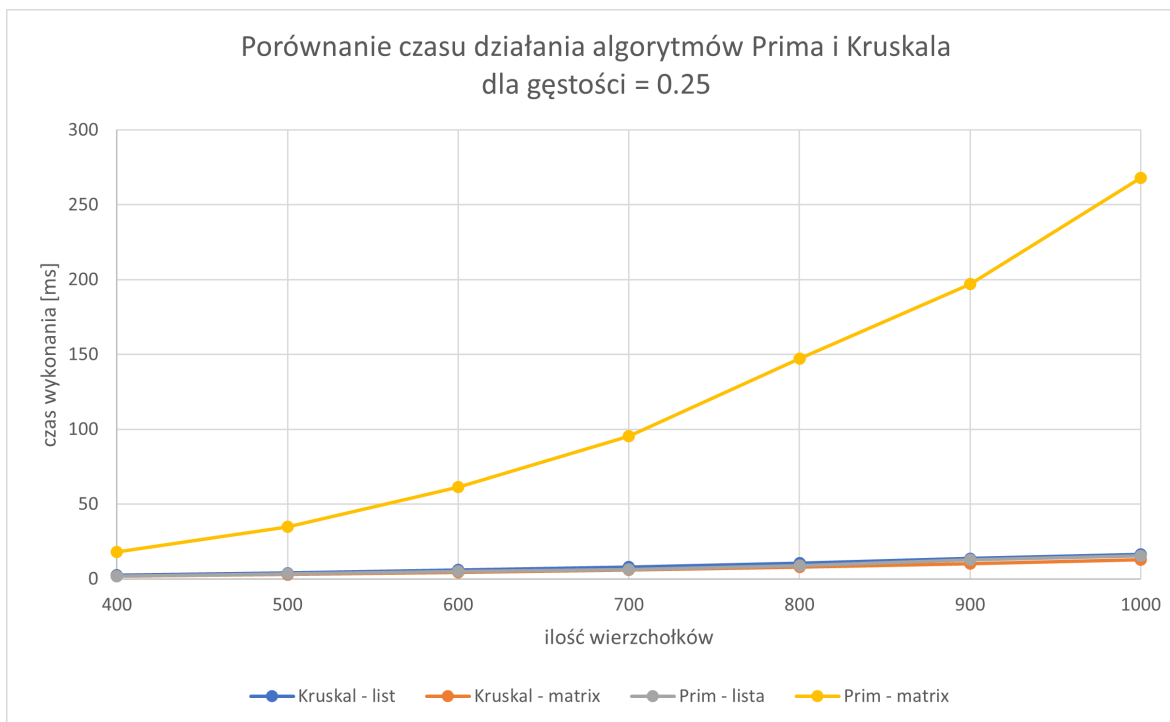


Figure 22: Czasy Prima i Kruskala – gęstość 25 %, lista vs macierz.

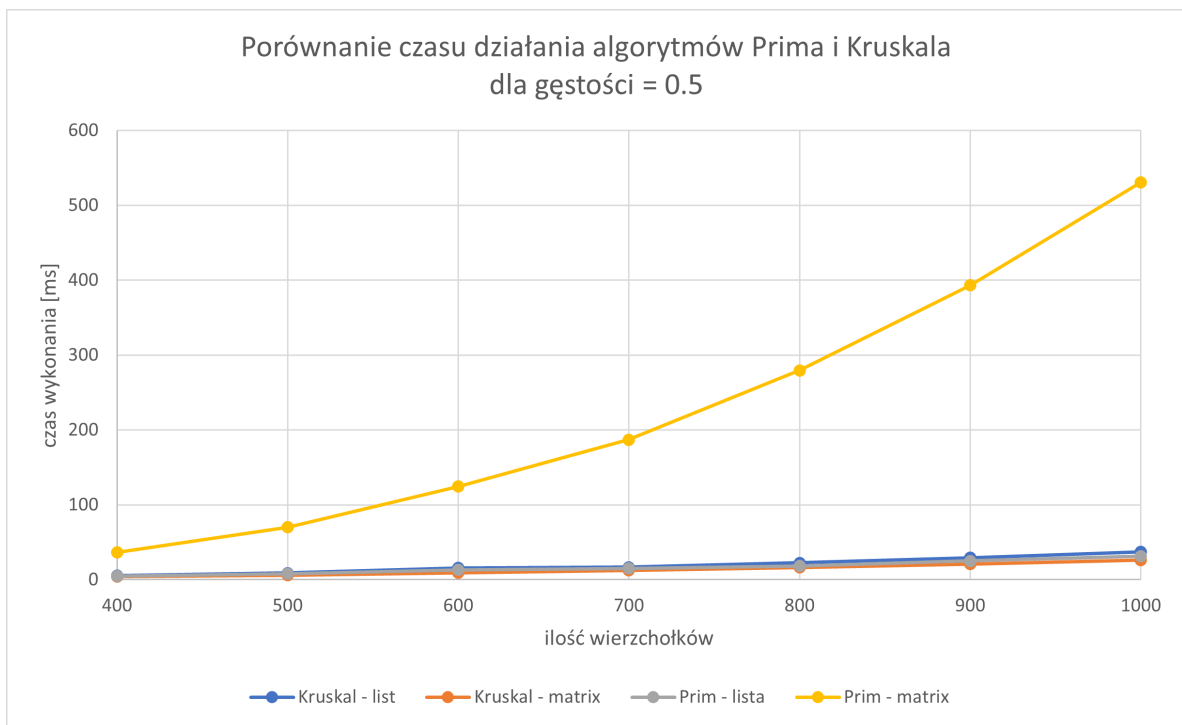


Figure 23: Czasy Prima i Kruskala – gęstość 50 %, lista vs macierz.

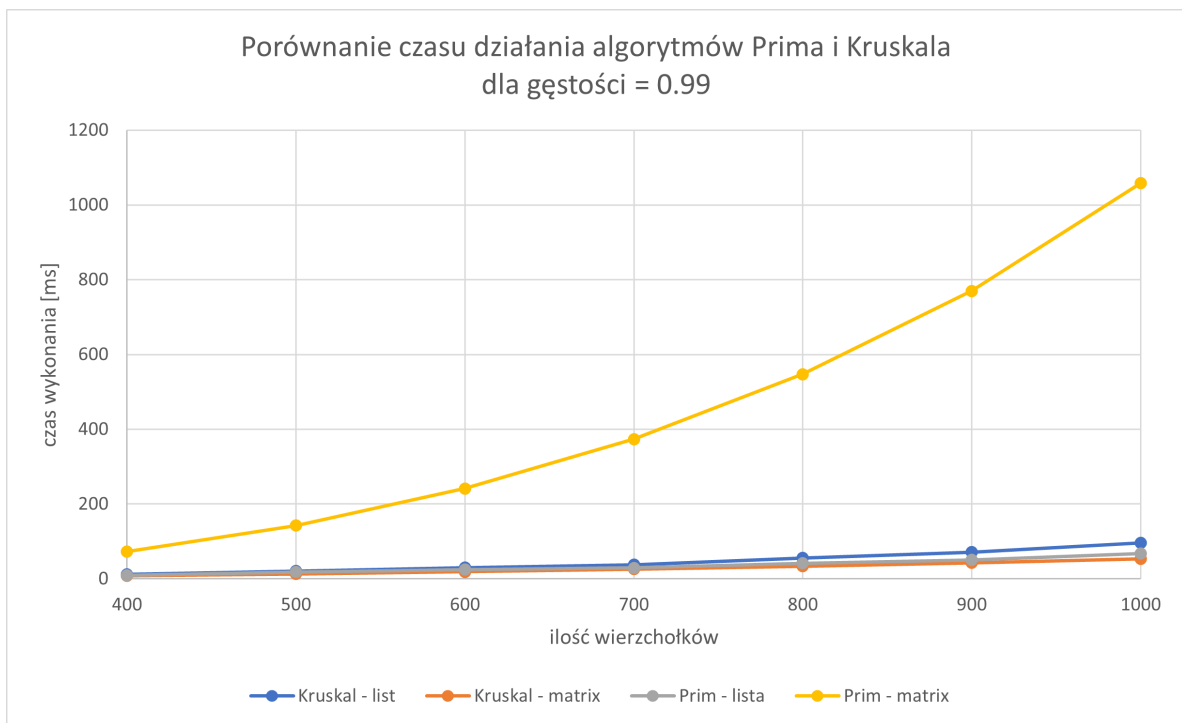


Figure 24: Czasy Prima i Kruskala – gęstość 99 %, lista vs macierz.

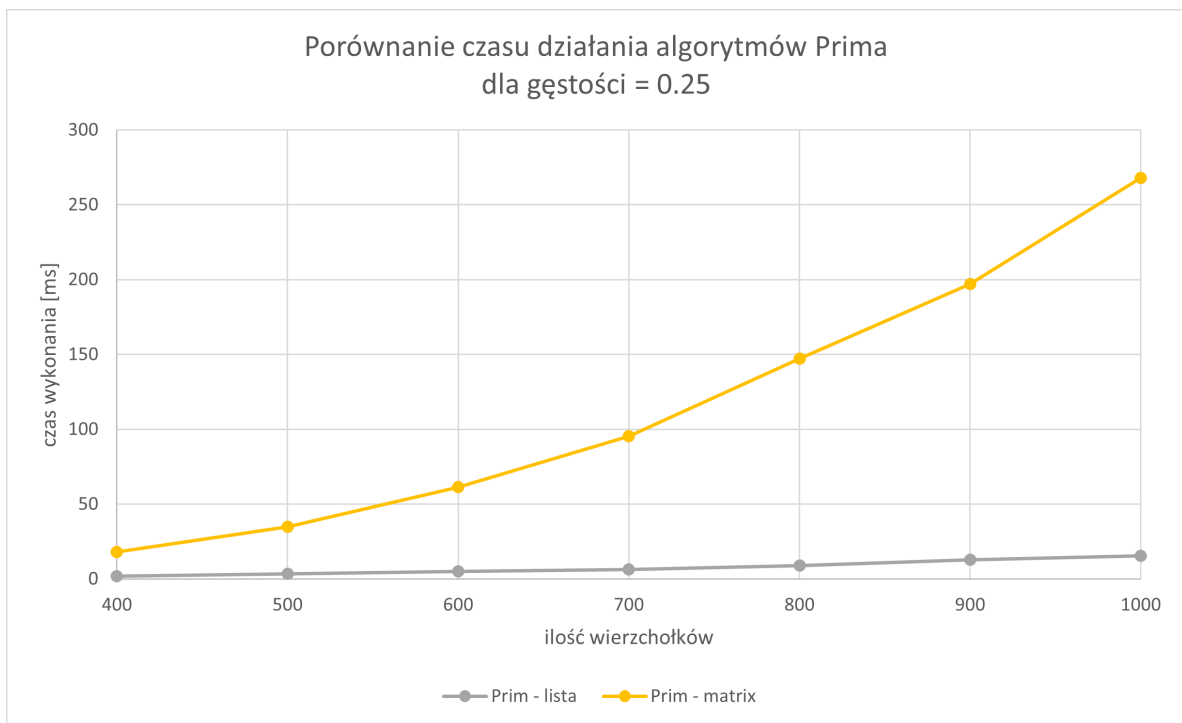


Figure 25: Prim – gęstość 25 %, lista vs macierz.

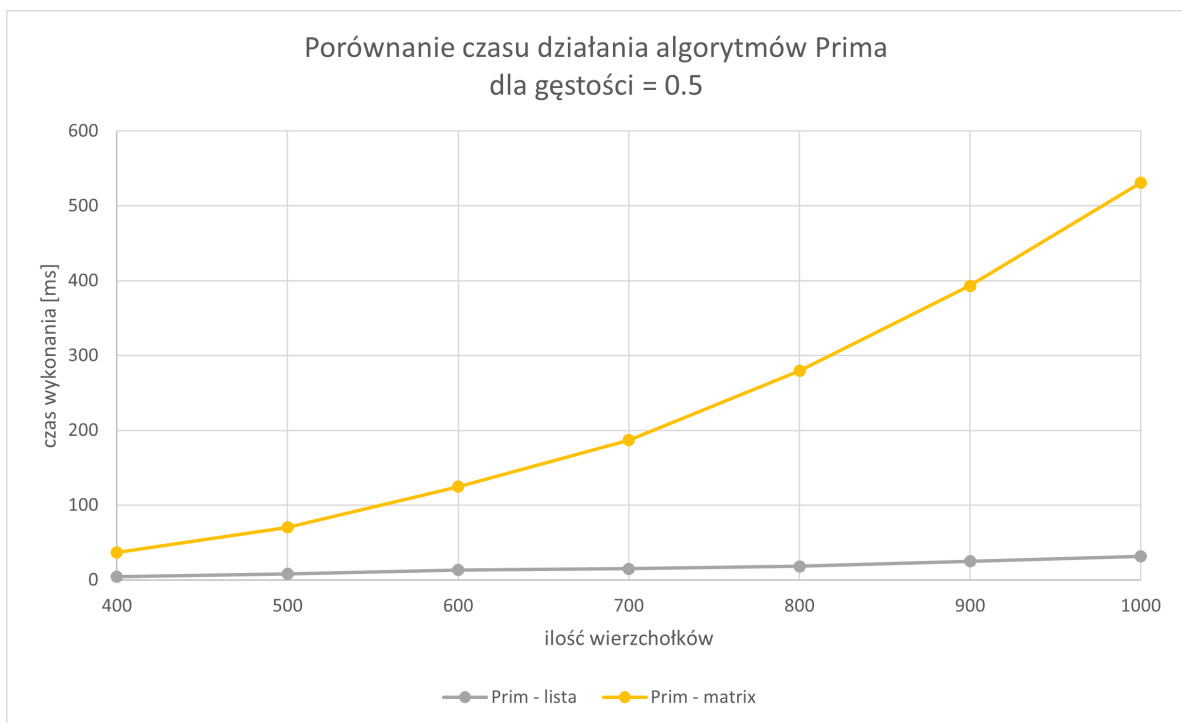


Figure 26: Prim – gęstość 50 %, lista vs macierz.

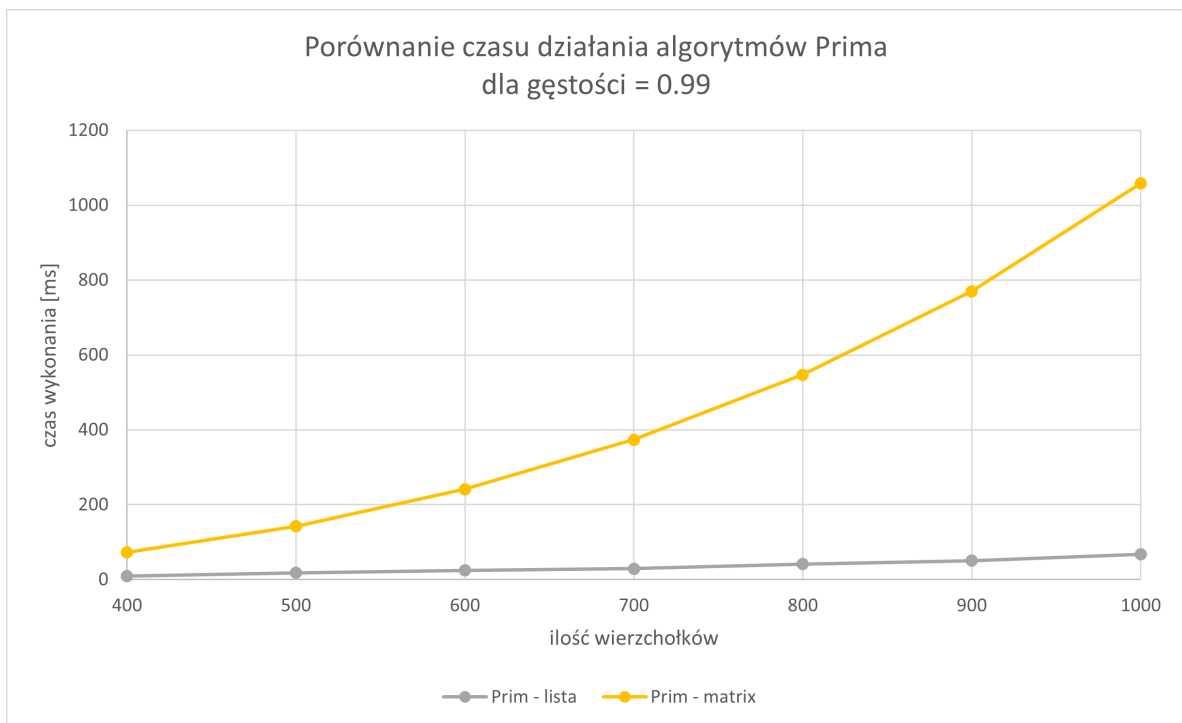


Figure 27: Prim – gęstość 99 %, lista vs macierz.

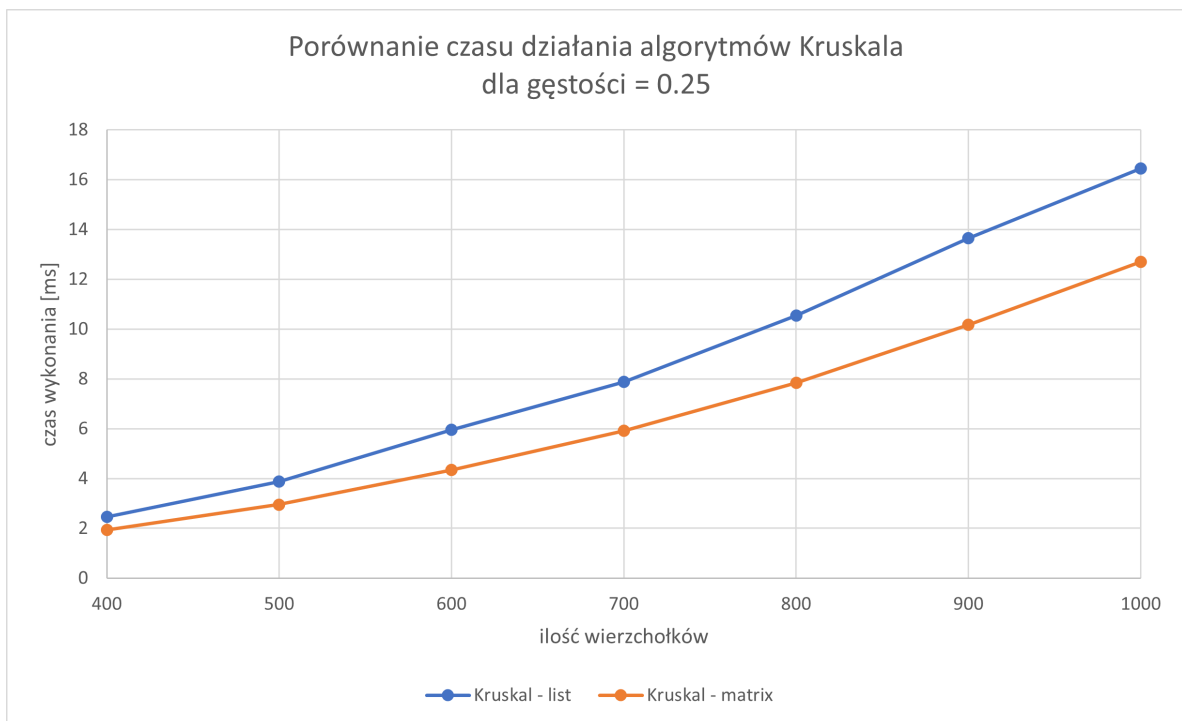


Figure 28: Kruskal – gęstość 25 %, lista vs macierz.

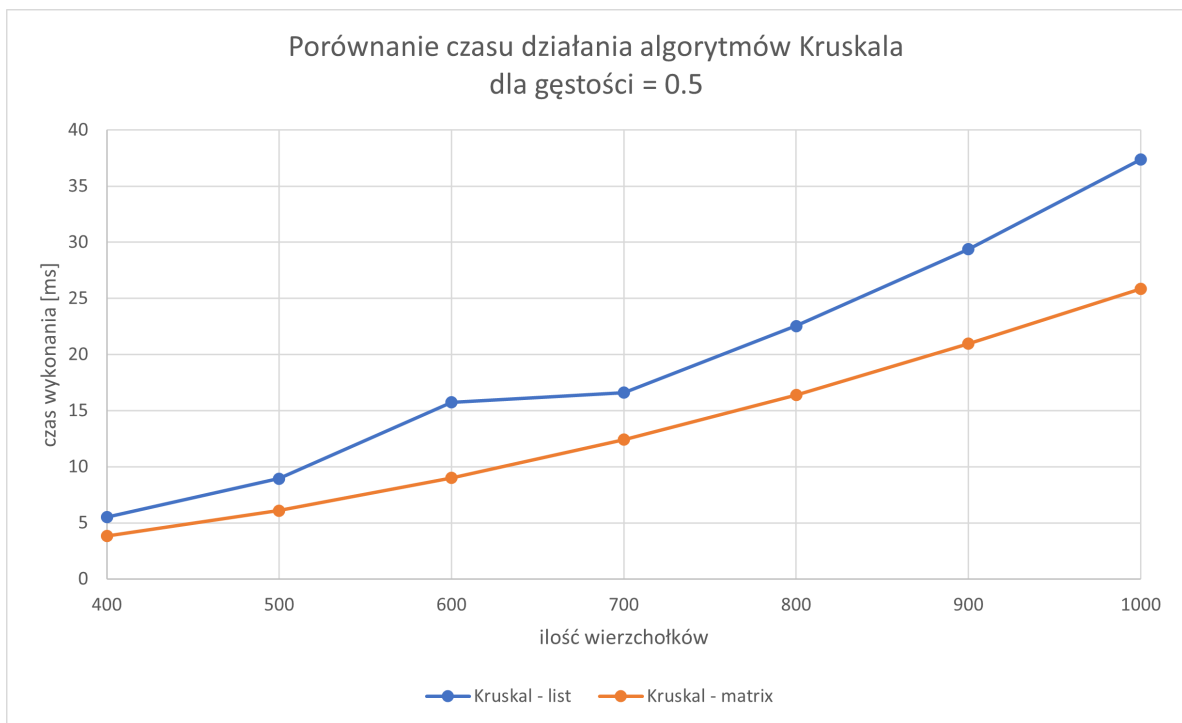


Figure 29: Kruskal – gęstość 50 %, lista vs macierz.

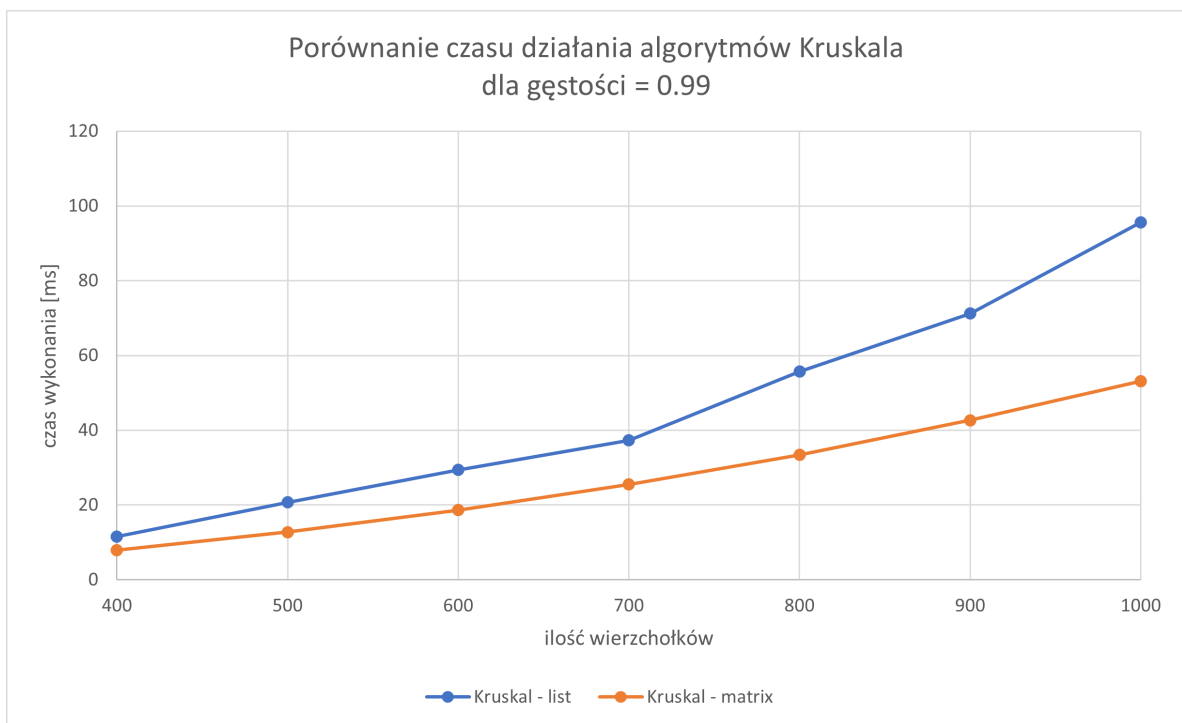


Figure 30: Kruskal – gęstość 99 %, lista vs macierz.

4.4 Wnioski

Algorytm Prima na reprezentacji listy sąsiedztwa zgodnie z teorią wykazał się znacznie krótszym czasem wykonania od macierzowej reprezentacji.

W przypadku algorytmu Kruskala wystąpiły rozbieżności pomiędzy złożonościami teoretycznymi a uzyskanymi w porównaniu obu reprezentacji. W wersji macierzowej Kruskal okazał się zauważalnie szybszy od wersji listowej. O różnicy stanowią dodatkowe trzy tablice które w macierzowej strukturze na bieżąco zapisują kolejno wierzchołek początkowy, końcowy oraz wagę. Dzięki temu nie trzeba za każdym razem skanować całej macierzy tylko wystarczy liniowo skopiować krawędzie i jednorazowo posortować.

Podsumowując algorytm Prim okazał się najszybszy jeżeli graf jest w postaci listowej, jednak dla grafu w postaci macierzy incydencji jego wydajność drastycznie spada i staje się najwolniejszym spośród algorytmów MST rozpatrywanych w eksperymencie. Algorytm Kruskala okazał się bardzo stabilnym rozwiązaniem niezależnie od reprezentacji grafu.

5 Literatura

Źródła

- [1] *Bellman-Ford algorithm* — *Wikipedia*. URL: https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm.
- [2] Thomas H. Cormen, Charles E. Leiserson, and Ronald L. Rivest. *Wprowadzenie do algorytmów*. MIT Press / WNT, 1997.
- [3] *Dijkstra's algorithm* — *Wikipedia*. URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
- [4] GeeksforGeeks. *Prim's Minimum Spanning Tree (MST) | Greedy Algo-5*. URL: <https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>.
- [5] GeeksforGeeks. *Time and Space Complexity of Dijkstra's Algorithm*. URL: <https://www.geeksforgeeks.org/time-and-space-complexity-of-dijkstras-algorithm/>.
- [6] *Kruskal's algorithm* — *Wikipedia*. URL: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm.
- [7] *Prim's algorithm* — *Wikipedia*. URL: https://en.wikipedia.org/wiki/Prim%27s_algorithm.